

Отчёт по лабораторной работе 3

Контейнеризация приложения средствами Docker

Милютин Никита, БР1.2

1. Тема

Контейнеризация микросервисного приложения бронирования столиков в ресторанах средствами Docker и Docker Compose.

2. Цель работы

Реализовать Dockerfile для каждого сервиса, написать общий docker-compose.yml и настроить сетевое взаимодействие между сервисами.

3. Состав приложения

В лабораторной работе собрана цельная версия микросервисного приложения в папке Labs/lab3. В состав входят:

Компонент	Порт	Назначение
API Gateway	3003	Единая внешняя точка входа
Auth Service	4001	Регистрация, вход, профиль
Restaurant Service	4002	Рестораны и фильтрация
Table Service	4003	Столики ресторанов
Reservation Service	4004	Создание и отмена бронирований
Notification Service	4005	Получение событий из RabbitMQ
RabbitMQ	5672, 15672	Брокер сообщений и web-интерфейс

4. Dockerfile

Для каждого сервиса создан отдельный Dockerfile:

- src/gateway/Dockerfile;
- src/services/auth/Dockerfile;
- src/services/restaurants/Dockerfile;
- src/services/tables/Dockerfile;
- src/services/reservations/Dockerfile;
- src/services/notifications/Dockerfile.

Каждый Dockerfile использует образ `node:22-alpine`, устанавливает зависимости через `npm ci`, копирует исходный код, выполняет `npm run build` и запускает нужный скомпилированный файл из `dist`.

5. Docker Compose

Общий файл `docker-compose.yml` поднимает все сервисы и RabbitMQ. Для взаимодействия создана сеть `restaurant-network`. Наружу публикуются только:

- 3003 для API Gateway;
- 5672 для AMQP RabbitMQ;
- 15672 для RabbitMQ Management UI.

Бизнес-сервисы доступны друг другу внутри Docker-сети по именам контейнеров: `auth`, `restaurants`, `tables`, `reservations`, `notifications`, `rabbitmq`.

6. Сетевое взаимодействие

API Gateway перенаправляет внешние HTTP-запросы во внутренние сервисы:

- `/api/auth/*` -> Auth Service;
- `/api/users/me` -> Auth Service;
- `/api/restaurants*` -> Restaurant Service;
- `/api/restaurants/:id/tables` -> Table Service;
- `/api/reservations*` -> Reservation Service;
- `/api/notifications` -> Notification Service.

Reservation Service при создании бронирования обращается к Restaurant Service и Table Service по внутренним адресам Docker-сети. После создания или отмены бронирования он публикует событие в RabbitMQ. Notification Service подписан на `reservation.*` и создаёт уведомления.

7. Запуск

Установка зависимостей:

```
npm install
```

Запуск всех контейнеров:

```
npm run dev
```

Остановка:

```
npm run down
```

8. Проверка

Проверка Gateway:

```
curl http://localhost:3003/
```

Получение ресторанов:

```
curl http://localhost:3003/api/restaurants
```

Создание бронирования:

```
curl -X POST http://localhost:3003/api/reservations \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer user-1" \
  -d '{"restaurant_id":1,"table_id":1,
"reservation_datetime":"2026-04-06T19:00:00.000Z","guest_count":2}'
```

Проверка уведомлений:

```
curl http://localhost:3003/api/notifications
```

9. Результат

В результате лабораторной работы приложение контейнеризировано. Для каждого сервиса создан Dockerfile, общий docker-compose.yml запускает всю систему одной командой, а сервисы взаимодействуют через внутреннюю Docker-сеть и RabbitMQ.